

1. Justificativa da Tecnologia Escolhida

Implementei o trabalho usando a linguagem Rust, por ser uma linguagem com que já tenho maior familiaridade e conhecimento.

Fiz uso do framework Axum e do cliente HTTP Reqwest, juntamente com o runtime Tokio que permite operações assíncronas.

A ideia de usar Rust veio por conta da segurança que o modelo de ownership do Rust ajuda a prevenir erros de I/O assíncrono. Além de ter uma performance do mesmo nível das linguagens C e C++, é muito mais seguro em termos de memória.

2. Decisões de Implementação Relevantes

Optei por seguir o modelo solicitado no enunciado, uma arquitetura de proxy baseada em prefixo de URL (CGI Proxy). E lidar com

3. HTTPS e Filtro de Conteúdo

A arquitetura implementada permite que o filtro de conteúdo funcione em sites HTTPS, porque a negociação TLS não está acontecendo entre o navegador e o site e sim o que o cliente está recebendo é um HTML diretamente do backend ou seja, é possível fazer o filter o alterações necessárias, porém a nível de Sistema Operacional, o navegador pede ao proxy para abrir um túnel TCP até o servidor de destino, então neste caso, toda o conteúdo é criptografado e não é possível alterar.

4. Aprendizados

Durante o desenvolvimento, eu tive alguns problemas que exigiram uma certa atenção, principalmente por conta de conceitos que eu nunca havia implementado, como a divisão entre cabeçalhos, a descompressão automática, já que o servidor envia o conteúdo com comprimento, e por conta do content-length limitado, o conteúdo estava sendo cortado, e entender e corrigir levou algum tempo.

5. Transparência no Uso de IA

Ferramentas de Inteligência Artificial baseadas em LLMs foram utilizadas ativamente no desenvolvimento deste trabalho.

- **Ferramentas utilizadas:** Antigravity AI (Assistente Integrado via LLM).
- **Como foram usadas:** A IA atuou como uma ferramenta de *Pair Programming*. Solicitamos esclarecimentos teóricos sobre manipulação de fluxos assíncronos (Bytes) no Axum/Reqwest e consultoria na lógica de parsing dos cabeçalhos. A documentação README também foi gerada utilizando LLM.
- **O que foi gerado/modificado:** A estrutura de roteamento e extração de URL foi construída manualmente, mas a IA ajudou a identificar e corrigir a função `forward_headers` para excluir automaticamente a lista de cabeçalhos *hop-by-hop*. Uma falha de XSS foi encontrada e corrigida usando o Claude Code para refatoração.
- **Aprendizado gerado:** A interação permitiu um aprofundamento mais rápido e direcionado nas APIs complexas do Rust (Axum e Tokio), além de promover debates valiosos que enriqueceram nosso entendimento sobre a teoria de proxies e TLS.